

Probabilistic Localization and State Estimation in Mobile Robotics: From Bayesian Filters to Motion Models

NECKER

June 10, 2026

Contents

1	Review of Probability and Bayes' Theorem	2
1.1	Fundamental Definitions	2
1.2	Bayes' Theorem	2
2	The Bayes Filter for State Estimation	2
2.1	The Markov Assumption (Complete-State Assumption)	3
2.2	Bayes Filter Algorithm	3
2.3	Formal Derivation of the Bayes Filter	5
3	Practical Implementations of the Bayes Filter	6
3.1	The Kalman Filter (KF)	6
3.2	Extended Kalman Filter (EKF)	7
3.3	Particle Filters (Monte Carlo Localization - MCL)	8
4	Action Models	8
4.1	Review: Differential Drive Kinematics	9
4.2	Odometry Model	9
4.3	Velocity Model	12
4.3.1	The Geometric Limit and Heading Drift	12
4.3.2	Evaluation of Probability $p(s_t s_{t-1}, u_t)$	12
4.3.3	Sampling in Particle Filters (Sampling)	13
5	Perception Models	14
5.1	Beam-Based Model	14
5.2	Likelihood Field Model (Scan Model)	15

1 Review of Probability and Bayes' Theorem

Probability is the foundation of reasoning under uncertainty, an intrinsic characteristic of autonomous robotics.

1.1 Fundamental Definitions

- **Probability of an event A :** $0 \leq P(A) \leq 1$.
- **Joint Probability:** $P(A, B)$ represents the probability that A and B occur simultaneously. If A and B are independent, then $P(A, B) = P(A)P(B)$.
- **Law of Total Probability:** For a discrete variable X , the marginal probability is $P(X) = \sum_y P(X|Y)P(Y)$. In the continuous case, it becomes an integral: $p(x) = \int p(x|y)p(y)dy$.

1.2 Bayes' Theorem

Bayes' Theorem provides a tool to update our beliefs (posterior probabilities) based on new evidence (sensory observations). Mathematically:

$$P(A|E) = \frac{P(E|A)P(A)}{P(E)} \quad (1)$$

Where:

- $P(A|E)$ is the **posterior probability** (diagnostic): what is the probability of cause A given the observed effect E ?
- $P(E|A)$ is the **Likelihood** (causal): if A were true, how likely would it be to observe the effect E ?
- $P(A)$ is the **Prior probability**: our initial belief about A before observing the evidence.
- $P(E)$ is the **Evidence**: it acts as a normalization factor.

2 The Bayes Filter for State Estimation

In autonomous robotics, the true state of a system (for example, the global pose of the robot $x_t = \langle x, y, \theta \rangle$) is never directly and perfectly accessible. Sensors and actuators are subject to thermal, systematic, and environmental noise. Therefore, the robot cannot know with absolute certainty where it is, but must estimate it based on the history of control actions $u_{1:t}$ (e.g., commanded wheel velocities) and sensor measurements $z_{1:t}$ (e.g., LiDAR readings or odometry).

To manage this uncertainty, the robot continuously maintains and updates a "belief" (*belief*) about the state, represented by a conditional probability distribution:

$$Bel(x_t) = p(x_t|z_{1:t}, u_{1:t}) \quad (2)$$

This expression indicates the probability that the robot is in state x_t , given all the information acquired from time 1 up to the current time t .

2.1 The Markov Assumption (Complete-State Assumption)

Directly computing $Bel(x_t)$ by taking into account an entire history that grows linearly over time (from $t = 1$ up to t) would entail an unmanageable computational burden ($O(t)$ at each iteration) and would require infinite memory.

To make the problem tractable and recursive, the fundamental **Markov Assumption** is introduced. It states that the present state x_t is a *complete and sufficient* summary of the entire past history. If we know the current state x_t , anything that happened before provides no additional useful information to predict the future.

WARNING: the markov assumption is applied to all possible robot positions calculated within the belief; this assumption serves only to reduce the amount of information needed to calculate the future state.

This assumption is composed of two fundamental conditional independences:

- **Measurement Independence (Perception Model):** The current measurement z_t depends *exclusively* on the current state x_t . Knowing past sensor readings does not help predict the current reading if the exact position of the robot is already known:

$$p(z_t | x_{0:t}, z_{1:t-1}, u_{1:t}) = p(z_t | x_t) \quad (3)$$

- **State Transition Independence (Motion Model):** The evolution toward the current state x_t depends *exclusively* on the immediately preceding state x_{t-1} and the control action u_t just executed. The specific path taken to reach x_{t-1} is irrelevant:

$$p(x_t | x_{0:t-1}, z_{1:t}, u_{1:t}) = p(x_t | x_{t-1}, u_t) \quad (4)$$

These relationships of dependence and independence can be visualized using the following graph (Hidden Markov Model - HMM):

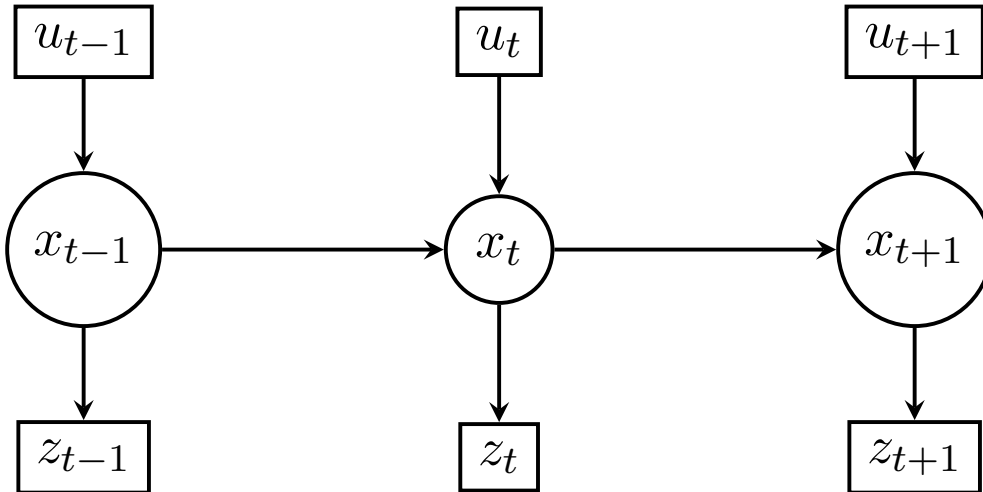


Figure 1: Hidden Markov Model (HMM) applied to mobile robotics. The states x are "hidden", while the measurements z and actions u are observable.

2.2 Bayes Filter Algorithm

The Starting Point (Time $t = 0$):

Before the robot performs any action, it must have an initial condition within its RAM. This is the initial belief, or absolute prior: $Bel(x_0)$.

- If the robot knows exactly where it is upon power-up (e.g., fixed starting coordinates), $Bel(x_0)$ will be a single peak (100% certainty on a single point).
- If the robot is turned on "blindly" at an unknown point (Global Localization), $Bel(x_0)$ will be a flat curve: all positions in space have the exact same probability.

The Recursive Loop (From $t - 1$ to t):

At this point, the loop begins. Suppose we are at a generic time step t . Only the final probability curve computed at the previous step remains saved **exclusively** in the robot's memory: $Bel(x_{t-1})$. All the rest of the past history has been deleted.

The robot performs two sequential steps to construct the new curve $Bel(x_t)$:

1. Phase 1: Prediction (Pushing the curve forward using Markov)

The robot has just executed a movement command u_t (e.g., "advance 1 meter"). Its goal now is to create a new temporary curve, called the *Predicted Belief* ($\overline{Bel}(x_t)$).

How does it create it? It leverages the Markov Assumption applied to the **Law of Total Probability**:

$$\overline{Bel}(x_t) = \int p(x_t|u_t, x_{t-1})Bel(x_{t-1})dx_{t-1} \quad (5)$$

- $p(x_t|u_t, x_{t-1})$ – Represents the conditional probability of reaching exactly the state x_t , given that the robot started from the specific point x_{t-1} and executed the command u_t . This term is directly responsible for "smearing" the uncertainty due to factors like wheel slippage or odometry drift.
- $Bel(x_{t-1})$ – It is the consolidated probability curve at the end of the previous cycle.

The physical result is that the entire old curve is literally "translated forward" in space and simultaneously "smeared" (widened), since physical movement always introduces new uncertainty (e.g., wheel slippage).

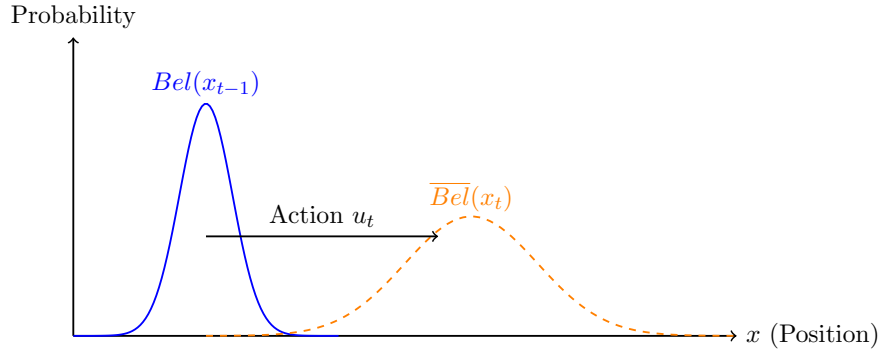


Figure 2: **Prediction:** The robot moves forward. The curve is translated toward the new position, but it flattens and widens (increase in variance) due to the intrinsic uncertainty of the movement.

Important: As soon as the new predicted curve $\overline{Bel}(x_t)$ has been drawn, the old one $Bel(x_{t-1})$ is definitively deleted from memory.

2. Phase 2: Correction (Cutting the curve using Sensors)

The newly created curve $\overline{Bel}(x_t)$ is, however, too uncertain (smeared). At this exact moment, the robot acquires a new reading from the sensors z_t (e.g., the LiDAR detects a wall at 20 cm).

The robot's goal is to use the probability curve of being in that position based on the sensor detection to shape the prior (if I am at $x = 60m$ and the wall was detected at $2.5m$, and the wall position is $x = 70$, then at that point the probability will be very low). It applies Bayes' Theorem by multiplying the predicted curve by the **Perception Model** (the result is a sort of interpolation between the two curves):

$$Bel(x_t) = \eta \cdot p(z_t|x_t)\overline{Bel}(x_t) \quad (6)$$

The robot asks a question for each point on the predicted curve: "If I were physically at this specific point, how likely would it be to see a wall at 20 cm?".

- If the answer is "very likely", the curve at that point is kept high or raised.
- If the answer is "impossible" (e.g., there are no walls nearby at that point on the map), the curve at that point is squashed to zero.

The operation drastically narrows the probability curve around positions that make logical sense with respect to the sensor reading. The parameter η acts as a simple mathematical **normalization factor**: it rescales the entire curve up or down so that the total underlying area equals exactly 1 (a mandatory condition for a probability distribution).

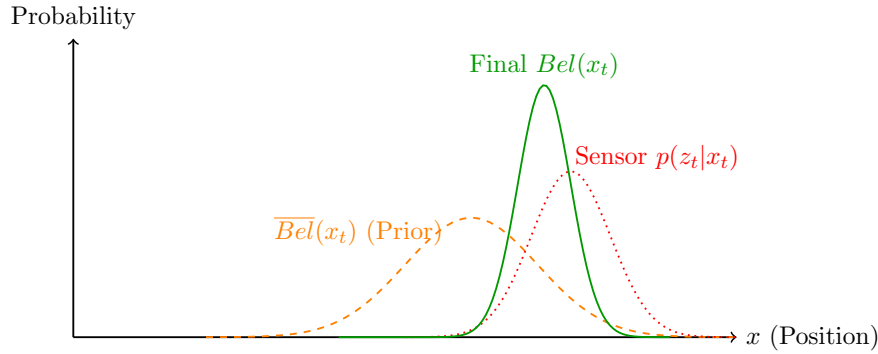


Figure 3: **Correction (Bayes Multiplication)**: The predicted curve (orange) is fused with the sensor curve (red). The result (green) is a compromise between the two: uncertainty is drastically reduced and the probability concentrates around the true position of the robot.

At the end of Phase 2, the curve $\overline{Bel}(x_t)$ is deleted. Only the new final *Belief* $Bel(x_t)$ remains in memory, and the robot is ready to wait for the next command to restart the identical loop at time $t + 1$.

NOTE THAT IN THE THEORETICAL MODEL THEY ARE NOT ALL GAUSSIAN CURVES (generally they represent the probabilities of finding ourselves in a given position, but if, for example, we have 3 identical doors we will have three peaks and not one like in a Gaussian), **!LOOK AT THE KALMAN FILTER!**

2.3 Formal Derivation of the Bayes Filter

The conceptual explanation just seen (Prediction and Correction) can be rigorously derived starting from the definition of the *belief* as a conditional joint probability. The goal is to compute and simplify the expression $Bel(x_t) = p(x_t|u_{1:t}, z_{1:t})$.

We proceed by applying the fundamental laws of probability:

1. **Application of Bayes' Theorem:** We treat the last measurement z_t as evidence and all prior history as background knowledge. The denominator $P(E)$ is absorbed into a normalization factor η :

$$Bel(x_t) = \eta \cdot p(z_t|x_t, u_{1:t}, z_{1:t-1}) \cdot p(x_t|u_{1:t}, z_{1:t-1})$$

2. **Markov Assumption (Perception Model):** Since the measurement z_t depends solely on the current state x_t and not on what happened in the past, the first term simplifies:

$$Bel(x_t) = \eta \cdot p(z_t|x_t) \cdot p(x_t|u_{1:t}, z_{1:t-1})$$

3. **Law of Total Probability:** To link the current time step to the previous one, we introduce the state x_{t-1} by integrating it over its entire domain inside the second term:

$$Bel(x_t) = \eta \cdot p(z_t|x_t) \int p(x_t|x_{t-1}, u_{1:t}, z_{1:t-1}) \cdot p(x_{t-1}|u_{1:t}, z_{1:t-1}) dx_{t-1}$$

4. **Markov Assumption (Motion Model) and Recursion:** We know that the evolution into state x_t depends only on x_{t-1} and the last action u_t . Furthermore, we notice that the last term of the integral $p(x_{t-1}|u_{1:t-1}, z_{1:t-1})$ is nothing but the exact definition of the *belief* at the previous step, $Bel(x_{t-1})$. Substituting, we obtain:

$$Bel(x_t) = \eta \cdot p(z_t|x_t) \int p(x_t|x_{t-1}, u_t) \cdot Bel(x_{t-1}) dx_{t-1}$$

We have thus obtained the final recursive formula of the Bayes Filter, which neatly splits the action-based update (the prediction integral) from the observation-based update (the external corrective multiplication).

3 Practical Implementations of the Bayes Filter

The Bayes Filter seen in its theoretical formulation is continuous and requires solving complex integrals over multi-dimensional state spaces. On a real computer (which has limited memory and computing power), this exact calculation is impossible.

For this reason, robotics utilizes approximate implementations, which are divided into two large families:

- **Parametric Filters** (such as the Kalman Filter) which assume fixed distributions (Gaussian).
- **Non-Parametric Filters** (such as Particle Filters) which use discrete sampling.

3.1 The Kalman Filter (KF)

The Kalman Filter is the perfect analytical solution to the Bayes Filter, valid only under three very strong assumptions:

1. The initial *belief* is a Gaussian distribution. If the robot were in a global localization scenario with 3 distinct peaks for 3 identical doors, the filter would fail instantly at $t = 0$. It wouldn't know how to describe those 3 peaks using only a mean and a covariance, because a Gaussian by definition has a single and unique peak (it is unimodale).
2. The motion model and the perception model are **strictly linear**.
3. The noise added by the motion (R_t) and by the sensors (Q_t) is purely Gaussian and zero-mean.

If these conditions are met, the probability remains an exact Gaussian forever. Consequently, the robot does not need to store a complex curve, but just needs two parameters (hence the name parametric filter):

- **The mean vector μ :** The estimation of the most probable position.
- **The covariance matrix Σ :** The uncertainty associated with the estimate.

The Kalman Filter equations transform the Bayes integrals into simple and extremely fast matrix multiplications:

$$\textbf{Prediction (Motion): } \bar{\mu}_t = A_t \mu_{t-1} + B_t u_t \quad (7)$$

$$\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t \quad (8)$$

$$\textbf{Correction (Sensors): } K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1} \quad (\textit{Kalman Gain}) \quad (9)$$

$$\mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t) \quad (10)$$

$$\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t \quad (11)$$

3.2 Extended Kalman Filter (EKF)

The limitation of the pure Kalman Filter is that real robots **are non-linear**. As we saw in kinematics, sines and cosines appear to calculate motion (if the robot moves tilted at a certain angle, it will displace by $\cos(\alpha)$ on x and by $\sin(\alpha)$ on y), and sensors measure Euclidean distances (square roots and arctangents). If we pass a Gaussian through a non-linear function (e.g., the velocity model $g(x)$, i.e., the real equation of motion), the curve deforms into a "banana" shape and is no longer a Gaussian. This would destroy the mathematics of the KF.

The **Extended Kalman Filter (EKF)** solves the problem by using a first-order Taylor approximation. The EKF *locally linearizes* the non-linear functions g (motion) by calculating its first derivatives, namely the **Jacobian Matrix G_t** , evaluated exactly at the mean point μ of the current estimate.

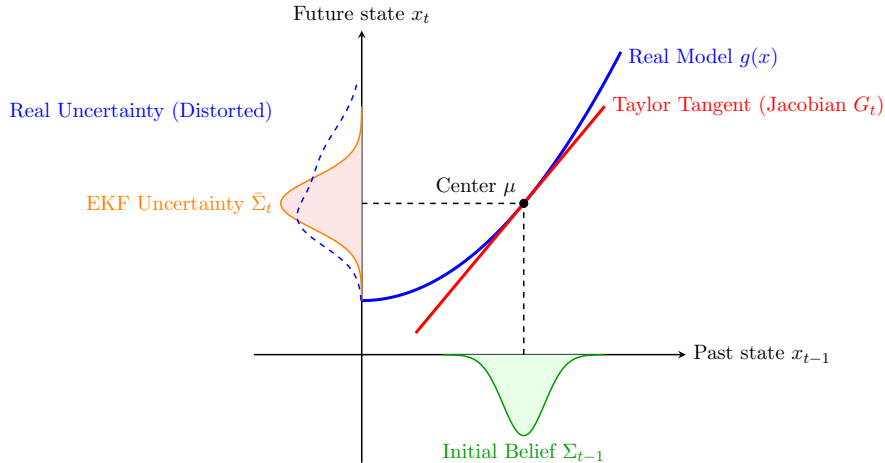


Figure 4: **The mechanics of the EKF:** The initial uncertainty (green) is projected not through the physical curve (blue), but through the Taylor tangent line (red). This trick forces the result (orange) to remain a perfect Gaussian, although it deviates slightly from the asymmetric reality (dashed blue curve).

As seen from the graph, projecting the initial Gaussian onto the red uncertainty line yields a perfect Gaussian (more elongated) on the y -axis; it will have its mean shifted (intersection

between old mean and line), and finally, this new Gaussian will be mapped back to the x -axis. **WARNING: this entire process replaces the very first part of the Bayes Filter algorithm when we use non-linear functions.**

3.3 Particle Filters (Monte Carlo Localization - MCL)

Gaussian filters (KF and EKF) manage unimodal distributions (the robot believes it is in only one place). If a robot is turned on blindly (*Global Localization*) or kidnapped and moved (*Kidnapped Robot Problem*), the EKF fails irreparably.

The **Particle Filter** is a non-parametric approach that represents the *Belief* not with mathematical formulas, but with a discrete swarm of thousands of *particles*. Each particle $x_t^{[m]}$ represents a single and complete hypothesis about the state of the robot.

The algorithm iteratively executes three magical phases inspired by natural selection:

1. **Sampling / Prediction (Sampling):** The robot acquires the odometric command u_t . For *each individual particle*, the Motion Model (e.g., Odometry Model) is applied, calculating the theoretical displacement and adding a random Gaussian error. The particles move and the "cloud" disperses.
2. **Importance Weighting / Correction (Importance Weighting):** The robot acquires the laser reading z_t . By querying the Perception Model (e.g., Likelihood Field), the system calculates how likely that reading is for each particle if the robot were located exactly there. This probability becomes the **importance weight** $w_t^{[m]}$ of the particle.
3. **Resampling:** This is the heart of the algorithm. A new set of particles is created from scratch. They are drawn "blindly" from the previous set using a rigged wheel of fortune (where the slots are proportional to the weights w).
 - Particles with high weights (that match well with the sensors) are drawn multiple times (they *clone* themselves).
 - Particles with low weights (off course or inside a wall) are ignored (they go *extinct*).

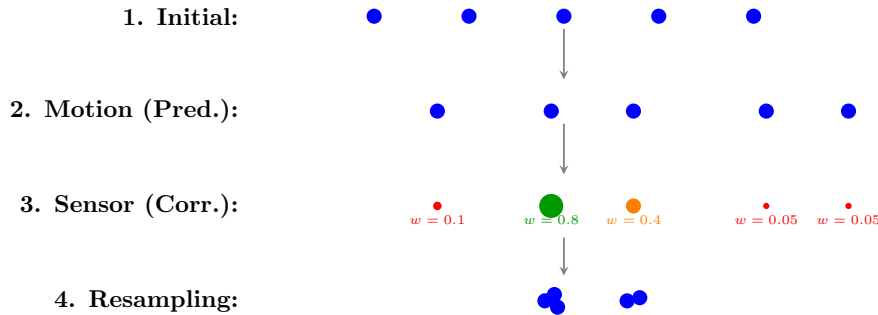


Figure 5: Lifecycle of the Particle Filter. Sensor measurements do not move the particles, but alter their "weight". Resampling eliminates incorrect hypotheses and concentrates computing power where the robot thinks it is.

4 Action Models

Motion models describe the probabilistic evolution of the robot's state. Their purpose is to compute $p(s_t | s_{t-1}, u_t)$, i.e., the probability that the robot is in pose s_t , knowing that it was in s_{t-1} and executed action u_t . In real implementations (like EKF or Particle Filters), these models are used for two main tasks:

- **Evaluation:** Given s_{t-1} , u_t , and an arrival hypothesis s_t , compute how probable this transition is.
- **Sampling:** Given s_{t-1} and u_t , randomly generate a possible arrival pose s_t by adding the correct noise.

4.1 Review: Differential Drive Kinematics

Before introducing probabilistic models, it is useful to recall the forward kinematic equations of a differential drive robot (the classic robot with two independent driving wheels, commonly used for *skid steering*). This helps us understand which physical levers the action u_t acts upon.

In a 2D planar environment, we define the robot's construction and control parameters:

- R : wheel radius.
- L : distance between the two wheels (wheelbase).
- ω_R, ω_L : angular velocities of the right and left wheels, respectively.

If the robot is at an orientation θ with respect to the global reference axis, the transition equations describing the linear velocities and angular velocity of its center (the pose x, y, θ) are given by:

$$\dot{x} = \frac{R}{2}(\omega_R + \omega_L) \cos(\theta) \quad (12)$$

$$\dot{y} = \frac{R}{2}(\omega_R + \omega_L) \sin(\theta) \quad (13)$$

$$\dot{\theta} = \frac{R}{L}(\omega_R - \omega_L) \quad (14)$$

In a discrete-time control system with time interval Δt , the next pose can be approximated by integrating these velocities:

$$x(t + \Delta t) \approx x(t) + \dot{x}\Delta t \quad (15)$$

$$y(t + \Delta t) \approx y(t) + \dot{y}\Delta t \quad (16)$$

$$\theta(t + \Delta t) \approx \theta(t) + \dot{\theta}\Delta t \quad (17)$$

This model, based exclusively on forward kinematics (*Forward Kinematics*), is purely **deterministic**. That is, it presupposes ideal conditions and a perfect world. However, if used for long-term navigation (*Dead Reckoning*), it is bound to fail: wheel slippage, friction asymmetries, and diameter variations introduce unpredictable errors.

To overcome this problem, mobile robots do not use pure kinematics, but overlay a statistical evaluation of the noise, giving rise to the probabilistic models we will examine now: the **Odometry Model** and the **Velocity Model**.

4.2 Odometry Model

The odometry model is used when the robot features proprioceptive sensors (they measure what actually happens during movement, such as encoders mounted on the wheels of a *differential drive*). Encoders measure the physical rotation of the wheels, allowing to compute an estimate (often affected by drift) of the movement performed.

Instead of working directly with wheel velocities, the action u_t is encoded by extracting the difference between the odometric pose read at time $t - 1$ and the one read at time t . Any movement on a 2D plane can be broken down into a geometrically equivalent sequence of three phases:

1. **First Rotation (δ_{rot1}):** The robot rotates on the spot to align itself with the direction of the destination.
2. **Translation (d):** The robot advances in a straight line toward the destination.
3. **Second Rotation (δ_{rot2}):** The robot rotates on the spot to assume the final desired orientation.

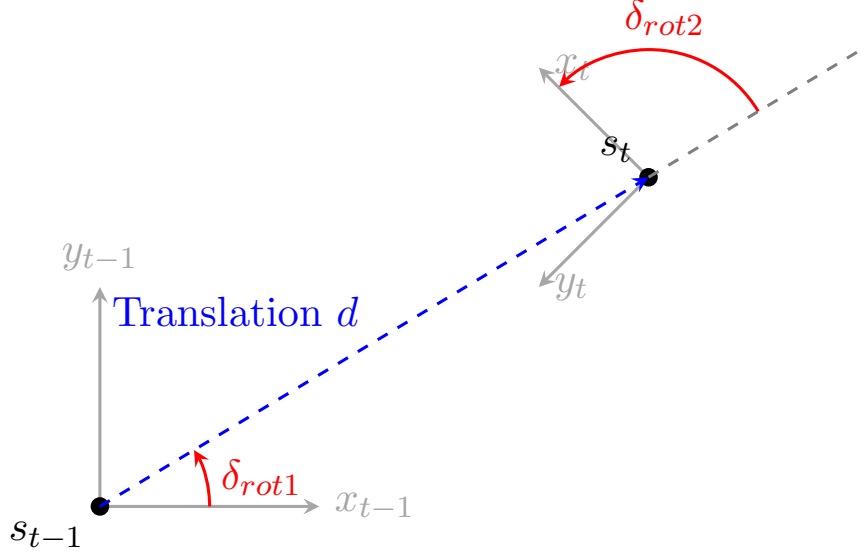


Figure 6: Odometric breakdown of movement into three phases (Rotation - Translation - Rotation).

Probability Evaluation in the Odometry Model:

The model does not generate a new pose, but answers the question: "Given the odometric displacement measured by the sensors $u_t = \langle \hat{\delta}_{rot1}, \hat{d}, \hat{\delta}_{rot2} \rangle$, what is the probability $p(s_t | s_{t-1}, u_t)$ that the robot is actually in the hypothetical pose s_t ?"

To calculate it, we proceed in two phases:

1. **Recovery of kinematic parameters ($rot \rightarrow tras \rightarrow rot$):**

We compute the exact analytical movement $\langle \delta_{rot1}, d, \delta_{rot2} \rangle$ that would have been mathematically necessary to transition from the previous pose s_{t-1} to the hypothetical pose s_t . We use this breakdown because it allows us to separate translation from rotations, thus being able to calculate the errors on each component independently.

2. **Statistical evaluation of errors:**

We compare the theoretically required movement $\langle \delta_{rot1}, d, \delta_{rot2} \rangle$ with the movement actually measured by the sensors $\langle \hat{\delta}_{rot1}, \hat{d}, \hat{\delta}_{rot2} \rangle$. The difference between the two represents the error Δ calculated on each component (e.g., $\Delta_d = d - \hat{d}$).

We evaluate the probability of committing this error using a Gaussian distribution $p_\sigma(\Delta)$ centered on zero (which represents the ideal condition of zero error). The logical and visual operation of this evaluation is based on two pillars:

- **The shape of the bell curve (Variance σ^2):** It depends on the magnitude of the measured movement and the noise parameters. A short odometric movement generates a narrow and tall bell curve (low uncertainty); a long movement generates a wide and squashed bell curve (high uncertainty).

- **The value retrieval (The error Δ):** The error acts as the x -coordinate on the graph. By entering the bell curve with our Δ , we read the corresponding height on the y -axis. A Δ close to zero will return a high probability, while a large Δ will cause us to fall on the "tails" of the bell curve, indicating an almost zero probability.
(that the pose indicated by the odometry is the true one)

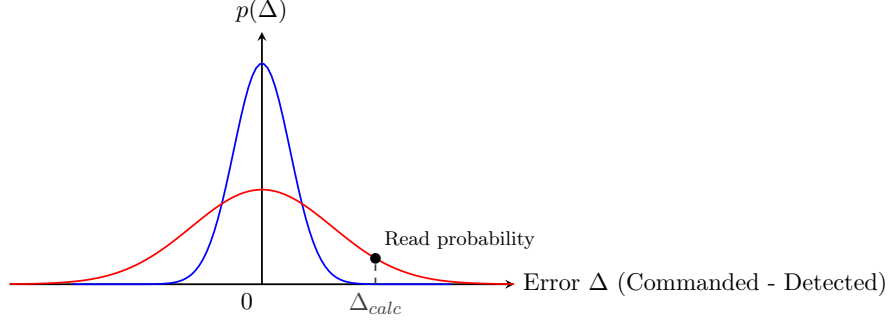


Figure 7: Evaluation of error probability. The width of the bell curve is defined by the measured movement and the parameters α ; the Δ determines the reading point.

Specifically, the variance (which defines how much the bell curve "widens") for each component is calculated by weighting the odometric readings through the intrinsic kinematic noise parameters of the robot ($\alpha_1, \alpha_2, \alpha_3, \alpha_4$):

- **Rotation error (σ_{rot1}^2 and σ_{rot2}^2):** Uncertainty increases drastically both for large steering maneuvers and during long translations (e.g., asymmetric wheel slippage on the ground). It is quantified as:

$$\sigma_{rot1}^2 = \alpha_1 \hat{\delta}_{rot1}^2 + \alpha_2 \hat{d}^2$$

$$\sigma_{rot2}^2 = \alpha_1 \hat{\delta}_{rot2}^2 + \alpha_2 \hat{d}^2$$

- **Translation error (σ_d^2):** Uncertainty grows if the robot travels long linear stretches (e.g., millimeter differences in wheel diameters), and is influenced by rotations. It is quantified as:

$$\sigma_d^2 = \alpha_3 \hat{d}^2 + \alpha_4 (\hat{\delta}_{rot1}^2 + \hat{\delta}_{rot2}^2)$$

4.3 Velocity Model

Unlike the odometry model, which relies on sensor measurements a posteriori, the **Velocity Model** is based on the control commands sent to the robot. The action $u_t = \langle v, \omega \rangle$ specifies the linear velocity v and angular velocity ω that the controller orders to maintain for a time interval Δt .

4.3.1 The Geometric Limit and Heading Drift

When sending constant velocity commands $\langle v, \omega \rangle$, theoretical kinematics dictates that the robot will move along a **perfect circular arc**.

If we tried to evaluate the probability $p(s_t | s_{t-1}, u_t)$ by simply calculating the unique circular arc that connects the starting position (x, y) to the arrival position (x', y') , we would clash with a structural limit of the model: the geometry of a perfect arc rigidly constrains the final orientation as well. The robot should arrive at the final pose with an angle θ' perfectly tangent to the arc. Any hypothetical pose s_t featuring an arrival angle that does not coincide with the tangent would mathematically receive a probability equal to zero, making the filter unusable in the real world (where bumps and slippage continuously modify the orientation).

To solve this problem, the model introduces a third parameter called **Heading Drift** ($\hat{\gamma}$). This value quantifies the unexpected extra rotation, allowing the model to tolerate and evaluate poses where the robot arrived at its destination but turned out to be "off-axis" with respect to the ideal trajectory.

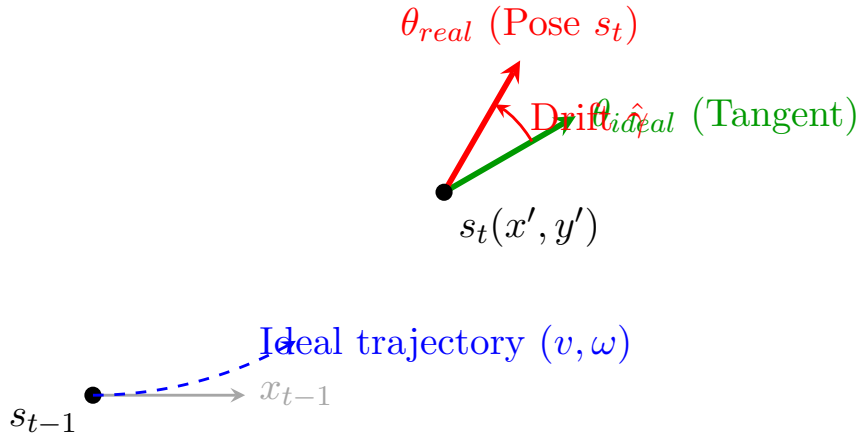


Figure 8: The introduction of Heading Drift ($\hat{\gamma}$) allows mathematical justification of final poses that are not perfectly tangent to the theoretical arc of movement.

4.3.2 Evaluation of Probability $p(s_t | s_{t-1}, u_t)$

During the filter update, to compute the probability that the robot is in the candidate pose s_t , the system operates as follows:

1. **Recovery of ideal velocities:** It analytically calculates which velocities $\langle \hat{v}, \hat{\omega} \rangle$ and which final drift $\hat{\gamma}$ would have been necessary to move exactly from s_{t-1} to s_t within time Δt .
2. **Calculation of discrepancies:** It compares the commanded velocities $\langle v, \omega \rangle$ with those ideally required $\langle \hat{v}, \hat{\omega} \rangle$, obtaining the errors.
3. **Statistical evaluation (Product of Gaussians):** It evaluates the three errors through three Normal distributions. The total probability is the product of the "scores" assigned

to the three components:

$$p(s_t|s_{t-1}, u_t) = p_v(v - \hat{v}) \cdot p_\omega(\omega - \hat{\omega}) \cdot p_\gamma(\hat{\gamma})$$

The variances σ^2 defining the width of the three Gaussian bell curves depend on the kinematic noise parameters of the robot ($\alpha_1 \dots \alpha_6$) and grow as the commanded velocities increase:

- **Variance on linear velocity:** $\sigma_v^2 = \alpha_1 v^2 + \alpha_2 \omega^2$
- **Variance on angular velocity:** $\sigma_\omega^2 = \alpha_3 v^2 + \alpha_4 \omega^2$
- **Variance on final drift:** $\sigma_\gamma^2 = \alpha_5 v^2 + \alpha_6 \omega^2$

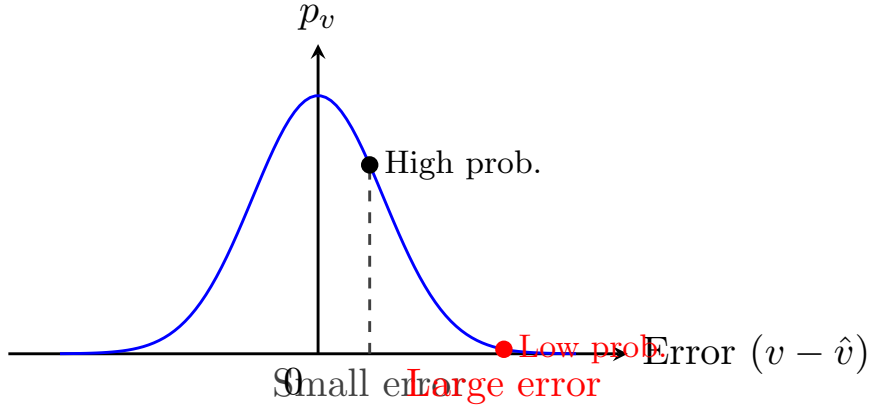


Figure 9: Example evaluation for the linear velocity component. The same bell-curve principle applies to angular velocity and drift $\hat{\gamma}$.

4.3.3 Sampling in Particle Filters (Sampling)

In the context of *Monte Carlo Localization*, the model is used in a generative sense to "move" the swarm of particles. Starting from pose s_{t-1} and receiving action $u_t = \langle v, \omega \rangle$, the new pose s_t is generated by applying random noise extracted from the respective Gaussian distributions:

1. The linear command is "noised": $\hat{v}_{noisy} = v + \text{Sample}(\mathcal{N}(0, \sigma_v^2))$
2. The angular command is "noised": $\hat{\omega}_{noisy} = \omega + \text{Sample}(\mathcal{N}(0, \sigma_\omega^2))$
3. A random angular offset is extracted: $\hat{\gamma}_{noisy} = \text{Sample}(\mathcal{N}(0, \sigma_\gamma^2))$

Finally, the particle is propagated for time Δt using forward kinematic equations with the modified velocities $(\hat{v}_{noisy}, \hat{\omega}_{noisy})$ and the drift angle $\hat{\gamma}_{noisy}$ is added to the final orientation, thereby obtaining the new hypothetical pose.

5 Perception Models

After estimating where it is based on movement (Prediction), the robot must use its exteroceptive sensors (cameras, LiDAR, sonar) to "look" at the environment and correct its estimate (Correction). The Perception Model has the task of computing the probability of the observation:

$$p(z_t|x_t, m) \quad (18)$$

This formula answers the question: *"If the robot were exactly in the hypothetical pose x_t within map m , how likely would it be to get the exact measurement z_t that the sensors have just returned?"*

The real environment is usually provided to the robot in the form of an **Occupancy Grid Map**, where space is divided into cells and each cell contains the probability of being occupied by an obstacle.

For rangefinder sensors like laser scanners (LiDAR), which return an array of distances (beams), there are two main mathematical models: the *Beam-based Model* and the *Likelihood Field Model*.

5.1 Beam-Based Model

The Beam-based model is physically very rigorous. For each single beam k emitted by the sensor, the algorithm performs a *raycasting* (traces a virtual ray) on map m (the map known to the robot) starting from pose x_t , to calculate distance z_k , which is the distance to the first obstacle on the map.

Once this is done, for each beam k , it calculates the position between pose x_t and the sensor detection by computing the ideal distance z_k^* where the object is located according to the sensor.

Finally, it calculates the difference between the two distances:

$$\Delta = z_k - z_k^* \quad (19)$$

However, real sensors are imperfect and the world is dynamic. The model takes into account **four independent types of error**, mathematically modeled and summed to obtain the final probability of a single reading z_k .

1. **Gaussian Measurement Error (p_{hit}):** This is the standard sensor noise. If the obstacle is at 3m, the sensor might read 3.02m or 2.98m. It is modeled with a Gaussian bell curve centered on the expected distance z_k^* .
2. **Unexpected Dynamic Obstacles (p_{short}):** The beam hits an object that is *not* on the map (e.g., a walking person or a moved chair). These obstacles always shorten the reading relative to z_k^* . It is modeled with an Exponential decay.
3. **Sensor Failure (p_{max}):** Sometimes laser light is absorbed (e.g., black surfaces) or reflected away (e.g., mirrors/glass). In these cases, the sensor receives no return signal and returns its maximum possible value by default (e.g., $z_{max} = 30\text{m}$). It is modeled with a single peak (Dirac Delta) on the maximum value.
4. **Random Measurements / Phantom Noise (p_{rand}):** Electromagnetic interference or multi-path reflections can generate totally random and meaningless readings. It is modeled with a flat Uniform distribution across the sensor's entire range.

The final probability for a single beam is the weighted sum (mixture) of these four distributions:

$$p(z_k|x_t, m) = w_{hit}p_{hit} + w_{short}p_{short} + w_{max}p_{max} + w_{rand}p_{rand} \quad (20)$$

Note: The sum of weights w must be equal to 1.

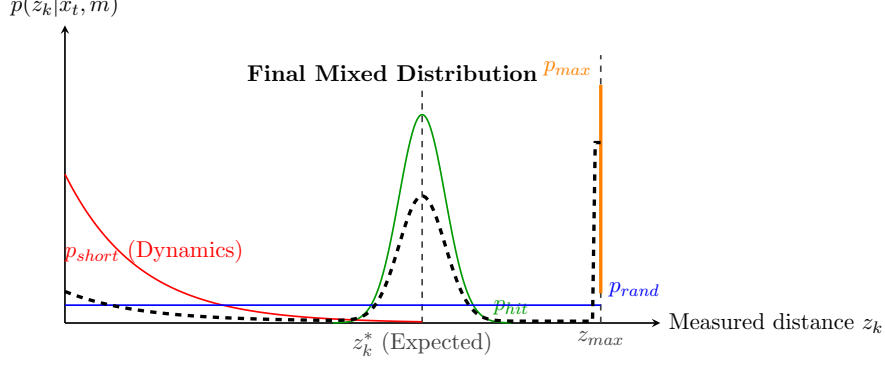


Figure 10: The four components of the Beam-based Model. The dashed black curve represents the overall probability that the sensor reads a specific distance z_k .

Ultimately, it uses Δ to find on the newly calculated Gaussian (which is centered on zero) the probability that the point under consideration is the true position of the robot.

The Problem with the Beam-based Model:

Despite being physically accurate, this model has two severe computational and robustness drawbacks. First, it requires simulating the *raycasting* on the grid for each particle and for each laser beam at every cycle (a huge CPU load). Second, in environments that are not perfectly smooth or have slight map defects, the model is overly strict, causing the probability to drop to zero in the presence of minor variations.

5.2 Likelihood Field Model (Scan Model)

To bypass the computational problems of the Beam-based model, modern filters use the **Likelihood Field Model**. Instead of calculating the entire path of the laser beam, this model evaluates exclusively the final useful result: *the exact point (end-point) where the laser hit something*.

The algorithm proceeds as follows:

1. **End-point projection:** Given the hypothetical pose of the robot x_t and the read distance z_k , it geometrically projects the space coordinates (x_{end}, y_{end}) where the obstacle was detected.
2. **Nearest neighbor search:** It checks on the global map m which static obstacle is closest to that point (x_{end}, y_{end}) by calculating the Euclidean distance d .
3. **Gaussian evaluation:** If the real obstacle is very close to the detected point (d small), the probability is high. If the end-point falls into a free area of the map far from any wall (d large), the probability drops. The whole mechanism is modeled with a zero-mean Gaussian whose variance is provided by the sensor manufacturer.

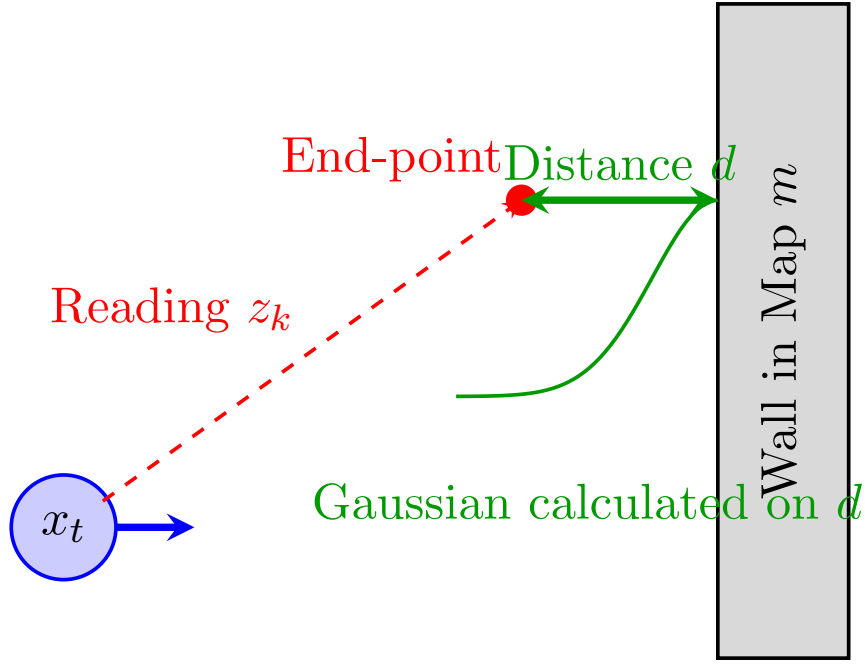


Figure 11: The Likelihood Field Model evaluates only the ray's end-point. The further the end-point is from an obstacle drawn on the map (distance d), the lower the probability assigned to that pose.

To further optimize calculation times, the map m is not queried at runtime. Instead, a special map is pre-computed in memory (a *Distance Transform*), in which each empty cell of the map already contains the pre-calculated distance to the nearest obstacle. In this way, updating the Likelihood Field becomes a trivial and instantaneous matrix memory access $O(1)$.